

CPU to GPU code migration strategies with C/C++ CUDA

GPU programming in computational engineering

Escuela de Doctorado 2025/26

University of Zaragoza

Sergio Martínez-Aranda

sermar@unizar.es

Mario Morales-Hernández

mmorales@unizar.es

Fluid Dynamics Technologies - I3A, University of Zaragoza



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

Module content

1. The SWE-2D system of equations
2. Variables and loops
3. Hands-on session in code migration

Module content

1. The SWE-2D system of equations
2. Variables and loops
3. Hands-on session in code migration

SWE-2D equations

Flow mass conservation

$$\frac{\partial h}{\partial t} + \frac{\partial q_x}{\partial x} + \frac{\partial q_y}{\partial y} = 0$$

Depth-averaged flow momentum

$$\frac{\partial q_x}{\partial t} + \frac{\partial}{\partial x} \left(\frac{q_x^2}{h} + \frac{1}{2} g h^2 \right) + \frac{\partial}{\partial y} \left(\frac{q_x q_y}{h} \right) = -g h \frac{\partial z_b}{\partial x} - g h \frac{n^2 \sqrt{u^2 + v^2} u}{h^{4/3}}$$

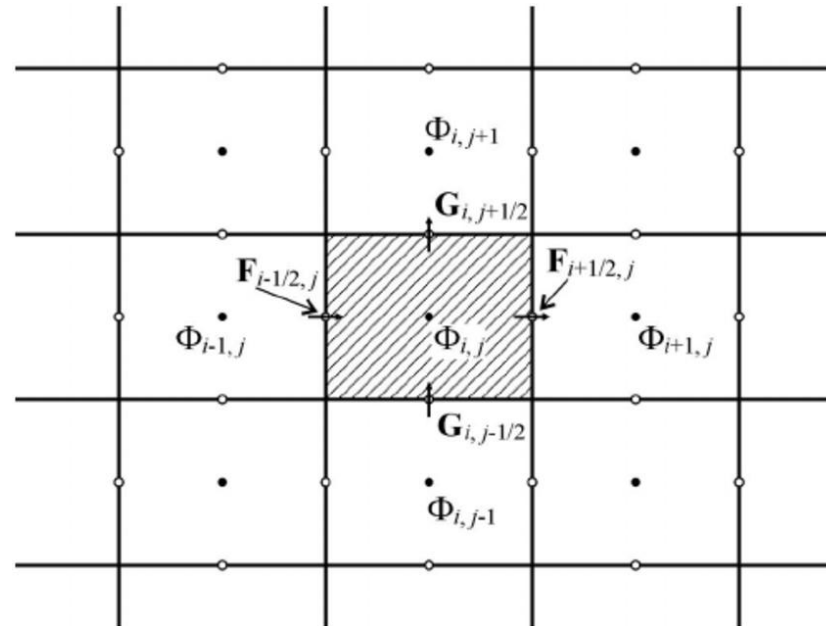
$$\frac{\partial q_y}{\partial t} + \frac{\partial}{\partial x} \left(\frac{q_x q_y}{h} \right) + \frac{\partial}{\partial y} \left(\frac{q_y^2}{h} + \frac{1}{2} g h^2 \right) = -g h \frac{\partial z_b}{\partial y} - g h \frac{n^2 \sqrt{u^2 + v^2} v}{h^{4/3}}$$

Numerical resolution

Conservative variables $U = \begin{pmatrix} h \\ q_x \\ q_y \end{pmatrix}$

Explicit first-order FV scheme

$$U_i^{n+1} = U_i^n - \frac{\Delta t}{A_i} \sum_{k=1}^4 \begin{pmatrix} \delta U_1 \\ \delta U_2 \\ \delta U_3 \end{pmatrix}$$



Module content

1. The SWE-2D system of equations
2. Variables and loops
3. Hands-on session in code migration

SWE-2D variables in CUDA code

- Spatial domain variables

```
int nX, nY;  
int nCells;
```

```
x = (double*) malloc( nCells* sizeof(double) );  
y = (double*) malloc( nCells* sizeof(double) );
```

- Rows & columns indexes

```
int i,j
```

- Global cell index

```
#define IDX(i, j, ncols) ((j) * (ncols) + (i))
```

- Time domain variables

```
double simTime;  
double t, dt, CFL;  
double Toutput;
```

SWE-2D variables in CUDA code

- **Conservative variables**

```
h = (double*) malloc( nCells* sizeof(double) );  
qx = (double*) malloc( nCells* sizeof(double) );  
qy = (double*) malloc( nCells* sizeof(double) );
```

- **Flow state variables**

```
ux = (double*) malloc( nCells* sizeof(double) );  
uy = (double*) malloc( nCells* sizeof(double) );
```

- **Distributed momentum sources**

```
n = (double*) malloc( nCells* sizeof(double) );  
zb = (double*) malloc( nCells* sizeof(double) );
```

SWE-2D variables in CUDA code

- Integrated & balance variables

```
double masWtn;  
double Qin, Qout;  
double massWerror;
```

SWE-2D variables in CUDA code

- Wall contribution containers

```
DU1 = (double*) malloc( nCells* sizeof(double) );  
DU2 = (double*) malloc( nCells* sizeof(double) );  
DU3 = (double*) malloc( nCells* sizeof(double) );
```

Workflow, functions & loops

- Read spatial domain
- Read initial conditions
- Read boundary conditions
- Read simulation setup

- Initialize distributed & integrated variables

```
h_initialize_variables(...){  
h_compute_initial_flow_variables(...){  
h_compute_water_mass(...){
```

```
for(ic=0; ic<nCells; ic++){  
...  
}
```

```
} //end function
```

Workflow, functions & loops

Time loop

```
while(t <= simTime) {  
    ...
```

- Wall flux calculations

```
h_compute_flow_time_step_2D(...){  
    h_compute_x_fluxes(...){  
        h_compute_y_fluxes(...){  
  
            for (int j = 0; j < nY; j++) {          // Rows  
                for (int i = 0; i < nX-1; i++) { // Columns  
                    int left = IDX(i,j,nX);  
                    int right = IDX(i+1,j,nX);  
                    ...  
                }  
            }  
        }  
    }  
}  
  
} //end function  
  
...
```

Workflow, functions & loops

Time loop

```
while(t <= simTime) {  
  ...
```

- Update cells

```
h_check_depth_positivity(...){  
h_update_cells_2D(...){
```

```
for(ic=0; ic<nCells; ic++){  
  ...  
}
```

```
} //end function
```

```
...
```

Workflow, functions & loops

Time loop

```
while(t <= simTime) {  
  ...
```

- Set cell interface wet-dry conditions

```
h_wet_dry_x(...) {  
h_wet_dry_y(...) {  
  
  for (int j = 0; j < nY; j++) {      // Rows  
    for (int i = 0; i < nX-1; i++) {    // Columns  
      int left = IDX(i,j,nX);  
      int right = IDX(i+1,j,nX);  
      ...  
    }  
  }  
  
} //end function  
  
...
```

Workflow, functions & loops

Time loop

```
while(t <= simTime) {  
    ...
```

- Set boundary values

```
h_set_west_boundary(...){  
h_set_east_boundary(...){
```

```
for(int j = 0; j < nY; j++) { //Rows  
    ...  
}
```

```
} //end function
```

```
...
```

```
h_set_north_boundary(...){  
h_set_south_boundary(...){
```

```
for(int i = 0; i < nX; i++) { //Columns  
    ...  
}
```

```
} //end function
```

Workflow, functions & loops

Time loop

```
while(t <= simTime) {  
  ...
```

- Check conservation errors
- Write output files

```
  ...
```

```
} //end time loop
```

Module content

1. The SWE-2D system of equations
2. Variables and loops
3. Hands-on session in code migration

SWE-2D code migration to C/C++ GPU

Task1

- Memory allocation and initialization
- CUDA kernel implementation

```
h_update_cells_2D(...) {...}  
h_set_<xxxx>_boundary(...) {...}
```

Task2

- CUDA kernel implementation

```
h_compute_x_fluxes(...) {...}  
h_compute_y_fluxes(...) {...}  
h_wet_dry_x(...) {...}  
h_wet_dry_y(...) {...}
```

- Memory transfer optimization

Tasks 3

- Race condition detection and resolution.

DU1, DU2, DU3

Macros for heterogeneous code management

```
#if __CUDACC__  
    #include <cuda.h>  
    #include <cuda_runtime.h>  
#endif  
  
#ifdef __CUDACC__  
    ...GPU code  
#else  
    ...CPU code  
#endif
```

Reduction for domain integrated variables

Total water mass in the domain

```
double masWtn;  
//Compute sum: massWt = area * sum(h)
```

Using cuBLAS (Basic Linear Algebra Subprograms) library:

```
#include <cusolver_v2.h>  
  
#if __CUDACC__  
...  
    cublasHandle_t handle;  
    cublasCreate(&handle);  
    cublasDasum(  
        handle,  
        nCells,  
        d_h,  
        1,  
        &massWtn);  
  
...  
#endif
```

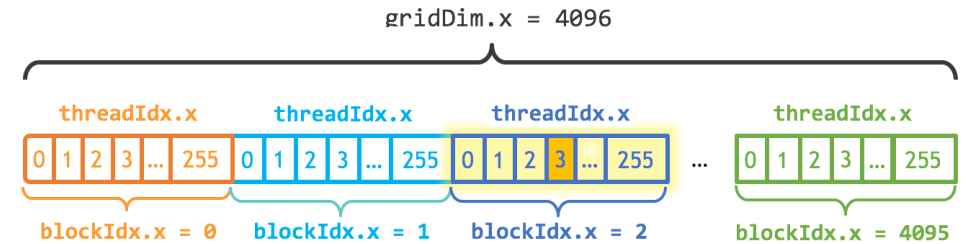
Param.	Memory	In/out	Meaning
<code>handle</code>		input	Handle to the cuBLAS library context.
<code>n</code>		input	Number of elements in the vector <code>x</code> .
<code>x</code>	device	input	<type> vector with elements.
<code>incx</code>		input	Stride between consecutive elements of <code>x</code> .
<code>result</code>	host or device	output	The resulting sum, which is set to <code>0</code> if <code>n <= 0</code> or <code>incx <= 0</code> .

Fix for race condition in cell contributions

Split wall contribution containers

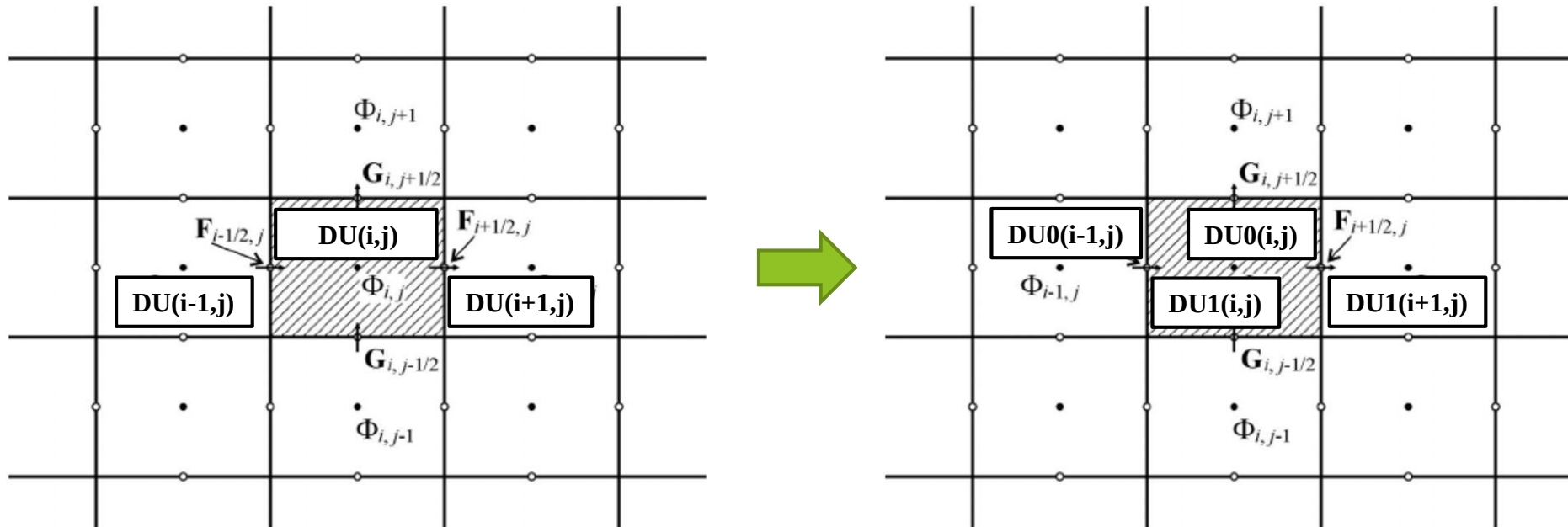
```
DU1_0 = (double*) malloc( nCells* sizeof(double) );
DU2_0 = (double*) malloc( nCells* sizeof(double) );
DU3_0 = (double*) malloc( nCells* sizeof(double) );
```

```
DU1_1 = (double*) malloc( nCells* sizeof(double) );
DU2_1 = (double*) malloc( nCells* sizeof(double) );
DU3_1 = (double*) malloc( nCells* sizeof(double) );
```



$$\text{index} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{index} = (2) * (256) + (3) = 515$$



References

NVIDIA developers, 2025. <https://developer.nvidia.com/>

CUDA Toolkit Documentation v12.8 Update 1, 2025. <https://docs.nvidia.com/cuda/>

<https://forms.gle/iCeLhMJnmqRq9Erm9>

